

Competitive Programming

Section 32: Combinatorics Under Modulo

Competitive Programming Course

2026-07-06

Table of contents

1	Why combinatorics needs preprocessing	3
2	Factorials modulo a prime	4
3	Inverse factorials	5
4	Binomial coefficients	6
5	Duplicate permutations	7
6	Stars and bars	8
7	Recognition table	9
8	C++ notes	10
9	Python notes	11
10	Common mistakes	12
11	Submission checklist	13

1 Why combinatorics needs preprocessing

Counting problems often ask for answers modulo 1,000,000,007. A direct formula such as:

$$C(n,k) = n! / (k! (n-k)!)$$

is mathematically correct, but not directly implementable with ordinary division under a modulus. Section 29 taught modular inverses; this section turns that into a reusable combinatorics engine.

2 Factorials modulo a prime

For a prime modulus MOD , precompute:

```
fact[i] = i! mod MOD
```

with:

```
fact[0] = 1  
fact[i] = fact[i-1] * i mod MOD
```

This takes $O(N)$ time for all $i \leq N$.

3 Inverse factorials

To compute combinations quickly, also precompute:

```
invfact[i] = inverse(i!) mod MOD
```

Because MOD is prime and $i < \text{MOD}$ in this section, $i!$ is not divisible by MOD, so the inverse exists.

First compute:

```
invfact[N] = fact[N]^(MOD-2) mod MOD
```

Then walk backward:

```
invfact[i-1] = invfact[i] * i mod MOD
```

Why? Since:

```
i! = i * (i-1)!
```

we have:

```
inverse((i-1)!) = inverse(i!) * i
```

4 Binomial coefficients

The number of ways to choose k objects from n is:

$$C(n, k) = n! / (k! (n-k)!)$$

Under the modulus:

$$C(n, k) = \text{fact}[n] * \text{invfact}[k] * \text{invfact}[n-k] \bmod \text{MOD}$$

This gives $O(1)$ per query after preprocessing.

5 Duplicate permutations

If all n objects are distinct, there are $n!$ permutations.

If some objects are equal, swapping equal objects does not create a new arrangement. For letter counts:

```
count[a], count[b], ...
```

the number of distinct rearrangements is:

```
 $n! / (\text{count}[a]! \text{ count}[b]! \dots)$ 
```

Modulo prime:

```
 $\text{fact}[n] * \text{product}(\text{inverse}(\text{fact}[\text{count}[\text{ch}]]) \bmod \text{MOD})$ 
```

Example:

```
"aab" has  $3! / 2! = 3$  arrangements:  
aab, aba, baa
```

6 Stars and bars

Count nonnegative solutions:

$$x_1 + x_2 + \dots + x_n = s$$

Think of s stars and $n-1$ separators.

Example for $n = 3$, $s = 4$:

`**|*|*`

means:

$$x_1 = 2, x_2 = 1, x_3 = 1$$

There are s stars and $n-1$ bars, so $s+n-1$ total positions. Choose which $n-1$ positions hold bars:

$$C(s+n-1, n-1)$$

7 Recognition table

Statement phrase	Formula
choose k from n	$C(n, k)$
rearrange letters with repeats	$n! / \text{prod}(\text{count}!)$
distribute identical objects among distinct boxes, zero allowed	$C(s+n-1, n-1)$
distribute positive objects among distinct boxes	$C(s-1, n-1)$ if $s \geq n$
path with only right/down moves on a grid	$C(\text{right}+\text{down}, \text{right})$

8 C++ notes

Use `long long` for multiplication:

```
fact[n] * invfact[k] % MOD * invfact[n-k] % MOD
```

Precompute up to the maximum value needed by all queries, not always the maximum constraint if the input is smaller.

9 Python notes

Python's built-in `pow(a, MOD-2, MOD)` is fast modular exponentiation.

Precomputing lists of size 2,000,001 is fine in this section.

10 Common mistakes

```
Using ordinary division under modulo.  
Forgetting fact[0] = 1.  
Precomputing only to max n when stars-and-bars needs n+s-1.  
Using k instead of n-k in one inverse factorial.  
Treating identical candies as distinct.  
Treating repeated letters as distinct.
```

11 Submission checklist

1. Is the modulus prime?
2. What maximum factorial index is needed?
3. Are all factorials and inverse factorials precomputed?
4. Does the formula match distinct vs identical objects?
5. Are zero cases like $k=0$, $s=0$, or $n=1$ handled naturally?